

Exercises - Debugging Techniques

In these 8 activities, you will practise reproducing a symptom, controlled benchmarking, interactive debugging with pdb, targeted instrumentation, diagnosing a hot path, investigating memory growth, fixing a root cause, and reporting evidence-based results.

STEP 1 OF 8

Reproduce and Define the Symptom

*Create a slow function and establish a repeatable baseline.
Describe the symptom in measurable terms, rather than saying
only that the program is slow.*

- Run the workload several times and record a typical elapsed time.
- Write a one-sentence performance problem statement, based on a measurable symptom rather than a feeling.
- Note the machine and conditions you tested under, so this baseline can be reproduced later.

```
step1_reproduce.py

import time

def process(values):
    total = 0
    for value in values:
        total += (value * value) % 97
    return total

values = list(range(300_000))

start = time.perf_counter()
result = process(values)
elapsed = time.perf_counter() - start

print("result:", result)
print("elapsed:", elapsed)
```

STEP 2 OF 8

Build a Controlled Diagnostic Benchmark

Create a benchmark that separates running the workload from reporting on it, and repeats the same operation several times. This becomes your controlled diagnostic environment.

- Keep the workload completely unchanged while diagnosing the problem - only the measurement code should differ.
- Explain why changing the workload between runs would make a before/after comparison meaningless.
- Identify sources of measurement noise, such as other running applications, CPU frequency scaling, or background processes.
- This snippet assumes process() from Step 1 is available in the same file.

```
step2_benchmark.py

import time

def benchmark(fn, runs=10):
    times = []
    result = None
    for _ in range(runs):
        start = time.perf_counter()
        result = fn()
        times.append(time.perf_counter() - start)
    return result, times

values = list(range(300_000))
result, times = benchmark(lambda: process(values))

print("min:", min(times))
print("avg:", sum(times) / len(times))
print("result:", result)
```

STEP 3 OF 8

Inspect Suspicious State with pdb

Use breakpoint() to stop inside a function the moment a suspicious condition occurs. The aim is to understand why the program is taking an unexpected path.

- At the breakpoint, inspect index, value, and total using p.
- Use where to see the full call stack, and c to continue execution once you understand what happened.
- Explain why conditional debugging (stopping only when a specific condition is met) is preferable to stopping on every iteration.



step3_pdb.py

```
def process(values):  
    total = 0  
    for index, value in enumerate(values):  
        if value < 0:  
            breakpoint()  
        total += value * value  
    return total  
  
values = [1, 2, 3, -4, 5]  
  
print(process(values))
```

STEP 4 OF 8

Add Targeted Timing and Call-Count Diagnostics

Instrument a suspicious function to determine whether the problem is excessive execution frequency, expensive individual calls, or both.

- Decide whether call frequency or per-call cost is the more important diagnostic clue here.
- Explain why instrumentation itself can distort very small timings.
- Consider how you would reduce the overhead of this instrumentation if the operation being measured became much faster.

```
step4_timing.py

import time

calls = 0

def expensive_transform(value):
    global calls
    calls += 1
    start = time.perf_counter()
    result = str(value).strip().lower()[::-1]
    elapsed = time.perf_counter() - start
    return result, elapsed

values = range(50_000)
total_time = 0.0

for value in values:
    _, elapsed = expensive_transform(value)
    total_time += elapsed

print("calls:", calls)
print("measured transform time:", total_time)
```

STEP 5 OF 8

Diagnose Repeated Work in a Hot Path

Investigate a function that repeatedly performs a membership test. Use reasoning, targeted counters, and measurement to determine whether the data structure is contributing to the slowdown.

- Estimate the cost of list membership testing at this scale.
- Form a hypothesis about an alternative data structure - but do not change the implementation yet.
- Explain why repeated list membership testing can become expensive as the list grows.

```
step5_membership.py

def count_matches(values, targets):
    checks = 0
    count = 0
    for value in values:
        checks += 1
        if value in targets:
            count += 1
    return count, checks

values = list(range(100_000))
targets = list(range(0, 100_000, 10))

result, checks = count_matches(values, targets)

print("matches:", result)
print("membership checks:", checks)
```

STEP 6 OF 8

Investigate Memory Growth

Use tracemalloc to identify exactly where memory allocation increases during repeated processing.

- Identify the allocation site from the printed statistics.
- Explain why the cached objects remain alive for the rest of the program.
- Distinguish legitimate retained data from an actual memory leak.
- Propose a bounded or streaming alternative, if the complete history is not actually needed.

```
step6_tracemalloc.py

import tracemalloc

cache = []

tracemalloc.start()
snapshot_before = tracemalloc.take_snapshot()

for i in range(50_000):
    cache.append({"id": i, "payload": str(i) * 5})

snapshot_after = tracemalloc.take_snapshot()

for stat in snapshot_after.compare_to(
    snapshot_before, "lineno",
)[:10]:
    print(stat)

print("retained objects:", len(cache))

tracemalloc.stop()
```

STEP 7 OF 8

Fix the Root Cause and Validate

Implement a targeted optimisation for the membership-test problem from Step 5. Preserve the result, and measure before and after.

- Explain the root cause in algorithmic terms (its Big-O cost), not just "it was slow."
- Explain why the optimisation preserves the required membership semantics.
- Compare the measured baseline and optimised times, and repeat the experiment if the difference looks small.

```
step7_fix.py

import time

def count_matches_list(values, targets):
    return sum(1 for value in values if value in targets)

def count_matches_set(values, targets):
    target_set = set(targets)
    return sum(1 for value in values if value in target_set)

values = list(range(200_000))
targets = list(range(0, 200_000, 10))

start = time.perf_counter()
a = count_matches_list(values, targets)
baseline = time.perf_counter() - start

start = time.perf_counter()
b = count_matches_set(values, targets)
optimised = time.perf_counter() - start

assert a == b

print("baseline:", baseline)
print("optimised:", optimised)
print("speedup:", baseline / optimised)
```

STEP 8 OF 8

Produce a Debugging and Optimisation Report

Create a concise report describing the complete investigation: the original symptom, the evidence collected, the root cause, the change made, correctness validation, and the performance improvement achieved.

- Fill in `baseline_seconds`, `optimised_seconds`, `speedup`, and `correctness_verified` using your own measured values from Step 7.
- State the test environment and workload size you used.
- State exactly what evidence supports your root-cause conclusion.
- State any limitations of your investigation, or possible follow-up work.

```
step8_report.py

report = {
    "symptom": "membership-heavy workload is slow",
    "baseline_seconds": None,
    "diagnostic_evidence": [
        "high repeated membership cost",
        "linear list membership",
        "large number of checks",
    ],
    "root_cause": "repeated O(n) membership searches",
    "change": "use a set for repeated membership tests",
    "optimised_seconds": None,
    "speedup": None,
    "correctness_verified": False,
}

print(report)
```